

國立台南大學資訊工程學系

116 級畢業專題期中進度報告

基於 Slurm 與 Kubernetes 架構下 AI 伺服器

GPU 工作負載智慧排程技術之研究

Intelligent GPU Workload Scheduling Techniques for
AI Servers under a Slurm-on-Kubernetes Architecture

專案編號：NUTN-CSIE-PRJ-116-016

組員： S11259043 鄭珽升

S11259033 蕭友翰

指導老師： 陳宗禧 教授

中華民國 115 年 6 月

國立台南大學資訊工程學系
畢業專題期中進度報告審定書

基於 Slurm 與 Kubernetes 架構下 AI 伺服器

GPU 工作負載智慧排程技術之研究

Intelligent GPU Workload Scheduling Techniques for
AI Servers under a Slurm-on-Kubernetes Architecture

學生 學號：S11259043 姓名：鄭珽升

學號：S11259033 姓名：蕭友翰

所提之報告內容符合本學系大學部畢業專題實作標準

指導教授：_____

系主任：_____

中華民國 115 年 6 月

成果貢獻摘要

本計畫已達成下列重要成果：

- 建立可實際運作之共享 GPU AI 工作負載彈性排程平台，整合 Slurm 批次排程與 Kubernetes 容器管理。
- 實現 CPU/GPU 計算節點自動彈性擴縮，依工作佇列長度動態啟停節點，有效降低硬體閒置浪費。
- 完成 GPU 資源切分機制 MPS，支援多個小型工作共享同一張 GPU，提升 GPU 使用效率。
- 實現風險敏感分佈式深度強化學習排程，模型可在真實系統中參與工作優先順序與資源放置決策。
- 建立系統監控與視覺化介面，即時呈現工作佇列、節點狀態、GPU 使用率及模型決策資訊。
- 完成平台穩定性修正（節點狀態同步、GPU 資源描述一致性），並通過實機驗證。

摘要

本研究以實驗室共享 GPU 主機常見的資源閒置、GPU 利用率不足與批次工作管理困難為背景，探討如何在容器化環境中建立兼具彈性與穩定性的 AI 工作負載排程平台。旨在整合 Slurm 的批次排程能力與 Kubernetes 的容器編排及彈性擴縮能力，讓使用者能以熟悉流程提交工作，同時由系統自動完成資源啟動、分配與監控。本專題提出以 Slurm-on-Kubernetes 為核心的共享 GPU 排程架構，並導入 MPS GPU 切分、節點自動擴縮、監控介面與深度強化學習排程插件等策略，以支援多工作共享 GPU 及智慧化資源放置。目前本平台已能在實機環境正確執行工作提交、GPU 分配、節點管理與 DRL 排程決策，但在模擬與實機 A/B 測試中，DRL 方法尚未穩定優於啟發式 score baseline。綜合而言，本研究完成了一套可運作的共享 GPU 彈性排程平台，並確認系統整合與實機驗證可行，後續仍需透過更完整的 baseline、RLPD 微調與真實資料訓練提升排程效能。

關鍵詞：GPU 資源排程、Kubernetes、Slurm、深度強化學習、彈性擴縮

目錄

成果貢獻摘要	I
摘要.....	II
目錄.....	III
第一章 動機與問題	1
1.1 研究動機.....	1
1.2 研究問題.....	1
第二章 文獻探討	2
2.1 高效能運算排程器 (Slurm)	2
2.2 容器化與 Kubernetes	2
2.3 GPU 資源共享技術.....	2
2.4 深度強化學習於資源排程.....	2
第三章 研究方法	5
3.1 系統整體架構.....	5
3.2 排程與資源分配流程.....	5
3.2.1 基本排程規則.....	5
3.2.2 智慧決策模型.....	6
3.2.3 指定資源配置機制.....	6
3.3 GPU 資源共享方法.....	6
3.4 模型訓練與評估方法.....	7
3.4.1 訓練與評估管線.....	7
3.4.2 評估指標.....	8
3.4.3 模擬配對基準測試.....	8
3.4.4 實機 A/B 測試	8
第四章 結果與討論	9
4.1 平台建置成果.....	9
4.1.1 平台建置與操作.....	9
4.1.2 監控與展示介面.....	9
4.2 評估結果.....	12
4.2.1 模擬環境評估.....	12
4.2.2 實機 A/B 測試結果	12
4.3 結論.....	12
第五章 參考文獻	14

第一章 動機與問題

1.1 研究動機

近年來，人工智慧研究越來越依賴 GPU 進行訓練、推論與資料處理。然而在一般研究室的共享主機環境中，常常面臨以下困境：

- 一張 GPU 只給一個工作使用，實際利用率不高。
- 沒有工作時，系統仍然維持大量資源待命，造成硬體閒置。

目前常見的系統大多只擅長其中一件事。容器平台如 Kubernetes 擅長自動啟動與回收資源，但對研究工作常見的 GPU 分配、批次排程與多使用者管理支援較弱；相對地，傳統高效能運算排程器 Slurm 雖擅長管理工作佇列與硬體資源，但對彈性擴縮與雲端式管理不夠方便。

因此，本專題希望結合兩者優點，建立一套既能保有研究者熟悉的工作提交流程，又能做到動態分配 CPU、GPU 與儲存資源的系統。進一步地，我們也希望導入機器學習方法，讓系統可以根據目前工作佇列狀態與硬體使用情形，自動做出更合理的資源分配決策。

1.2 研究問題

本計畫主要探討以下幾個問題：

- 如何在容器化叢集(Kubernetes)上建立一套適合研究工作的排程平台。
- 如何根據工作數量，自動擴縮 CPU 與 GPU 計算節點。
- 如何讓多個較小的 GPU 工作共享同一張顯示卡，以提升使用效率。
- 如何利用深度強化學習模型，協助系統決定哪些工作應優先執行，以及應使用哪些硬體資源。
- 如何在模型判斷不可靠時，讓系統自動回到較穩定的基本排程方式。

第二章 文獻探討

2.1 高效能運算排程器 (Slurm)

Slurm Workload Manager [1] 是目前高效能運算 (HPC) 叢集最廣泛使用的工作排程系統之一。它提供完整的工作提交、佇列管理、資源限額與帳務管理功能，並支援 GPU 資源分配。其以「節點」為基礎的資源模型非常適合研究工作環境，且已有廣泛的使用者社群與文件支援。然而 Slurm 原生設計以固定實體節點為主，對彈性擴縮與容器化整合的支援相對有限。

2.2 容器化與 Kubernetes

Kubernetes [3] 是目前最主流的容器編排平台，能夠自動管理容器的部署、擴縮與健康監控。Kubernetes 的 Cluster Autoscaler 可根據工作負載自動增減節點，非常適合需要彈性資源的場景。然而 Kubernetes 原生的排程器 (kube-scheduler) [10] 以服務導向設計為主，對批次工作、GPU 共享與研究工作特有的排程需求支援不足。因此有多項研究嘗試將 Slurm 與 Kubernetes 整合，以結合兩者優點。在 Slurm 與 Kubernetes 整合方面，既有研究已指出 HPC 與 cloud-native 系統的匯流可以同時取得批次排程語意與雲端彈性管理能力 [4]；實作層面也有 Slinky 等 Slurm-on-Kubernetes 方向的工具 [6]，以及 AWS ParallelCluster 等雲端 HPC 叢集部署方案 [8]，顯示此類架構已具有實務需求與發展基礎。

2.3 GPU 資源共享技術

NVIDIA 提供多種 GPU 資源共享技術，包括 Time-Slicing、Multi-Process Service (MPS) [12] 以及 Multi-Instance GPU (MIG)。MPS 允許多個 CUDA 程式同時共享一張 GPU 的運算資源；MIG 則在硬體層面將 GPU 切分成獨立的分區，提供更強的隔離性。本系統目前採用基於 MPS 的方式，讓多個較小的工作共享同一張 GPU，以提升整體使用效率。

除了本系統採用的 MPS 之外，近年也有研究探討在 MIG 等機制上進行動態重新分割與能源效率最佳化 [20]；這類方法能提供較強隔離性，但也需要額外處理硬體支援、分割粒度與工作遷移成本，因此本專題目前先以部署門檻較低的 MPS 作為共享 GPU 的主要實作方式。

2.4 深度強化學習於資源排程

近年來已有多項研究將深度強化學習應用於資源排程問題，包括資料中心工作排程、雲端資源分配等。然而多數工作採用期望值導向 (expectation-based) 的獎勵函數，僅最佳化平均表現，忽略了不確定情境下的風險。Ma X., et al. [14]

提出的 Distributional Soft Actor-Critic for Risk-Sensitive RL (RDSAC) 透過分佈式 critic 估計每個動作的 reward 分布，並結合 Soft Actor-Critic (SAC) [16] 的最大熵探索框架來實現風險敏感決策。相較於僅看平均值的方法，RDSAC 能夠同時考量好情況與壞情況，根據風險偏好做出更穩健的選擇，非常適合共享 GPU 平台等對穩定性有較高要求的場景。

在 AI 與 GPU 工作排程相關研究中，已有工作探討 CPU-GPU 任務於 AI 服務流程中的排程問題 [18]、異質 edge GPU 環境的模型排程技術 [19]、雲端 GPU 資源動態配置 [21]，以及具多資源感知與 live migration 支援的 GPU 排程方法 [22]；綜合性的 GPU scheduling survey 也指出，GPU 排程需要同時考量利用率、延遲、公平性、隔離性與工作負載異質性 [23]。

RDSAC 採用的分佈式價值估計與 Implicit Quantile Network (IQN) 相關，IQN 可用分位數方式描述回報分布，而不只估計單一平均 Q 值 [17]，因此適合延伸到本研究關注的尾部延遲與風險敏感排程問題。

Soft Actor-Critic (SAC) 採用 scalar twin-Q 與離散 categorical actor。在最大熵框架下，SAC 同時最佳化期望累積回報與策略熵，使策略兼顧效能與充分探索：

$$J = \sum_t E[r_t + \alpha H_t]$$

其中 α 為可自動調整的溫度參數， H_t 為步驟 t 的策略熵 $H(\pi(\cdot|s_t))$ 。Critic 採用雙 Q-network（取最小值），並以 soft 貝爾曼目標更新：

$$y = r + \gamma(1 - d)V(s')$$

$$V(s') = \sum_{a'} \pi(a'|s') [\min Q(s', a') - \alpha \ln \pi(a'|s')]$$

$$L_Q = \sum_i E[(Q_i(s, a) - y)^2]$$

策略（actor）最小化以下目標，在每個合法動作上對熵正則化（鼓勵探索）與 Q 值最大化取得平衡：

$$L_\pi = E_s \sum_a \pi(a|s) [\alpha \ln \pi(a|s) - \min Q(s, a)]$$

將此一風險敏感分佈式 SAC（簡稱 RDSAC）作為核心排程演算法。我們把 GPU 工作排程建模為馬可夫決策過程 (MDP)：狀態 s 為佇列中前 16 個待排工作的特徵與叢集 MPS 使用狀況，動作 a 為「選擇某个工作並指定其節點與 GPU 放置」或不放置 (no-op)，獎勵則以負的工作完成時間 (JCT) 為主。

RDSAC 的特點是不只估計回報的平均值，而是估計整個回報的機率分布。它在最大熵框架下，把 soft return 拆成兩個分布：獎勵回報分布 Z_R 與熵回報分布 Z_H ，兩者各以隱式分位數網路 (IQN) 表示，並以 quantile Huber 損失回歸。某個狀態-動作的綜合價值為兩者期望值之和：

$$Q(s, a) = E[Z_R(s, a)] + \alpha E[Z_H(s, a)]$$

其中 α 為自動調整的溫度參數，控制探索程度。為了讓策略對「壞情況」更敏感，RDSAC 在策略目標中對獎勵分布 Z_R 套用風險扭曲函數 ρ 。本專題採用條件風險值（CVaR）：只取分布下尾 β 比例的平均，使策略偏好最差情況較不嚴重的放置決策：

$$\rho_\beta[Z] = \frac{1}{\beta} \int_0^\beta F_Z^{-1}(\tau) d\tau$$

其中 β 越小越保守； $\beta = 1$ 時 CVaR 退化為一般平均。由於排程動作是離散的，因此將原始論文連續控制的高斯 actor 改寫為顯式的 categorical actor。策略以最小化下式的目標更新，對每個合法動作同時權衡熵（鼓勵探索）與風險調整後的回報：

$$J_\pi(s) = \sum_a \pi(a|s) [\alpha \ln \pi(a|s) - \rho(Z_R(s, a)) - \alpha E[Z_H(s, a)]]$$

直覺上，當風險模式設為 mean ($\beta = 1$) 時等同風險中立、只看平均；設為 cvar 時則會主動避開可能造成長尾延遲（如 straggler、冷啟動 worker、執行時間離群）的放置決策，特別適合對穩定性要求高的共享 GPU 平台。

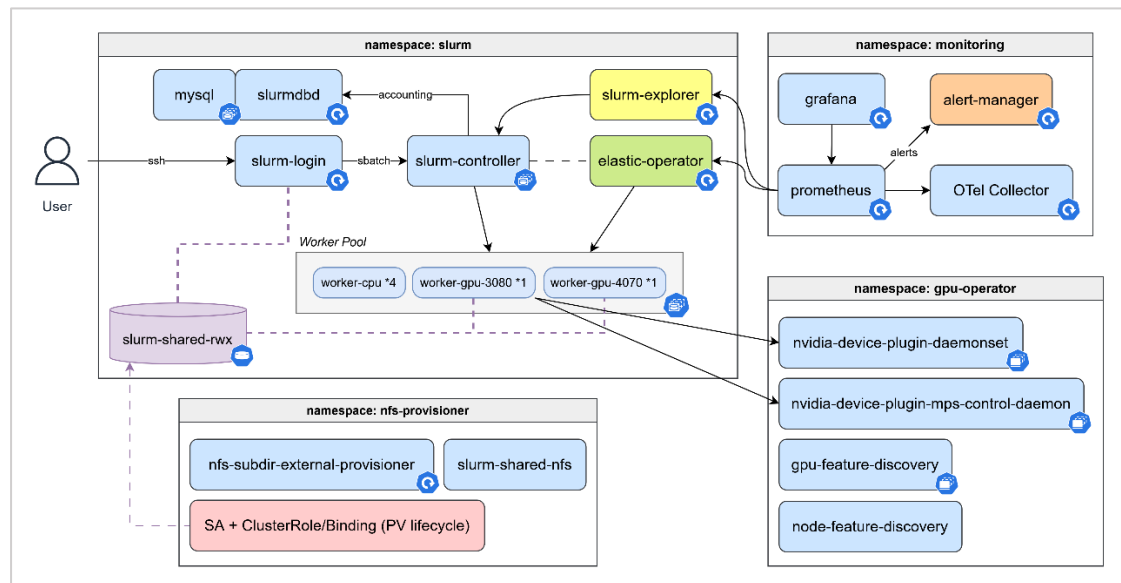
第三章 研究方法

3.1 系統整體架構

本系統的核心概念是把「批次工作排程」與「彈性資源管理」結合起來。使用者仍然以熟悉的方式提交工作，而系統在背後自動處理工作排序、節點啟動、GPU 分配與執行監控。如下圖 1 所示，整體系統包含下列幾個部分：

在使用者環境管理方面，登入節點搭配 Lmod [9] 等模組系統管理 CUDA、編譯器與實驗套件版本，讓使用者維持接近傳統 HPC 叢集的操作習慣，同時把實際執行環境交由後端容器與 Kubernetes 節點管理。

- (1) 工作提交入口：提供使用者提交與查詢工作的介面。
- (2) 中央排程控制：負責接收工作、維護佇列、決定優先順序。
- (3) CPU 與 GPU 計算節點：實際執行工作的運算資源。
- (4) 彈性控制模組：根據佇列長度與執行狀態，自動啟動或回收計算節點。
- (5) 智慧決策模組：利用強化學習模型協助判斷工作排序與資源配置。
- (6) 監控與追蹤模組：記錄系統狀態、工作過程與資源使用情況。



【圖 1】系統整體架構圖

3.2 排程與資源分配流程

目前系統的決策流程分成三層，實作成 Slurm 插件 [2]：

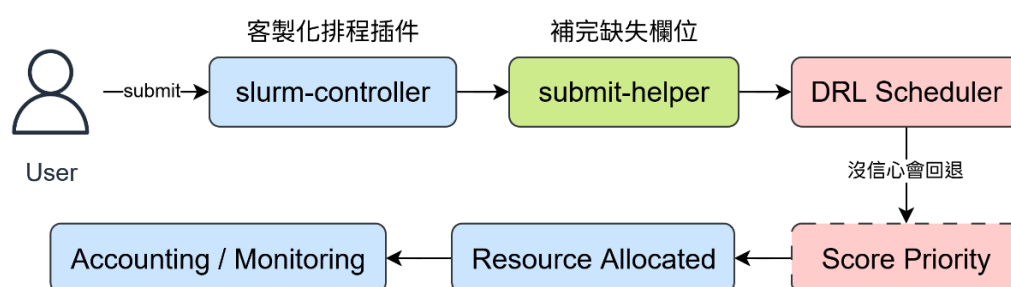
3.2.1 基本排程規則

當使用者提交工作後，系統先補齊必要資訊，並根據幾個簡單穩定的規則

計算工作的優先程度。這些規則主要考慮：工作是否適合目前剩餘的 GPU 資源、是否造成資源碎片化、以及較短的工作是否應優先執行。此層作為系統的保底機制，即使智慧模型失效，系統仍可正常運作。

3.2.2 智慧決策模型

由於目前 Kubernetes v1.35 的新功能 workload-aware scheduling [5] 仍在 alpha 版。故自行加入深度強化學習模型，讓系統可根據「目前有哪些工作在等待」、「哪些節點正在執行」、「GPU 尚有多少可分配空間」等資訊，進一步判斷哪一個工作應優先處理、是否需要額外提高優先順序，以及工作比較適合放到哪一個節點上。整體決策流程如下圖 2 所示：



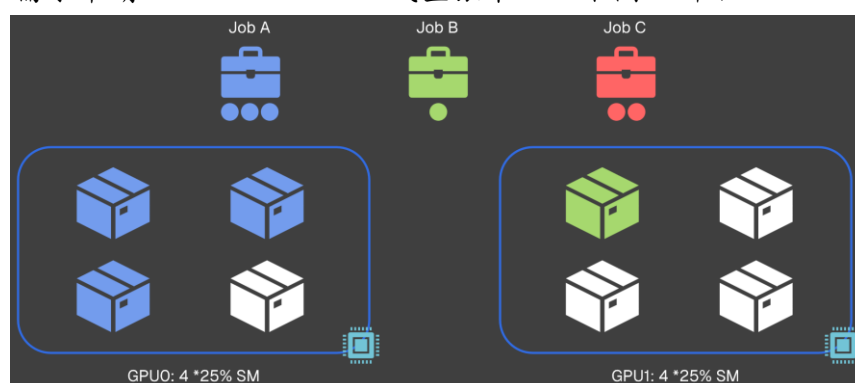
【圖 2】工作提交後的決策流程圖

3.2.3 指定資源配置機制

除了影響工作的先後順序外，系統也能先暫時保留工作，再根據模型決定其應放到哪一台節點上。這樣的做法讓模型的判斷不只停留在「哪個工作先跑」，而是更進一步參與「工作要放在哪裡跑」的決策。

3.3 GPU 資源共享方法

為提升利用率，可以透過 MPS (Multi-Process Service) 讓多個較小的工作共享同一張 GPU。以目前的系統設定為例，一張 GPU 可切成多個可分配單位，使用者可依需求申請 0.25、0.50、0.75 或整張卡，如下圖 3 所示：



【圖 3】GPU 資源切分示意圖

3.4 模型訓練與評估方法

將比較三種排程方式，採完全相同的訓練配方，僅調整指令參數，以區分各機制的貢獻：

- 啟發式 Score：以加權線性組合公式計算工作優先級，分數越高代表該工作越值得優先排程。五個因子及對應權重說明如下表 1，預設不啟用 topology 和 runtime fit 兩項。

【表 1】啟發式因子與係數定義

因子	係數	定義
MPS fit	$\alpha = 0.40$	$mps_req/100 \in [0, 1]$ 。沒給=1.0，超過=0.0。
VRAM fit	$\beta = 0.20$	$(1 - (fit_tier - req))/max_tier \in [0, 1]$ 。 依 job 需求選最小可用 VRAM tier，沒給=0.5。
Topology	$\gamma = 0.00$	保留欄位，固定回傳 0.5
Fragmentation	$\delta = 0.20$	$4(x - 1), x = mps_req/100$ 。mps_req=0 或滿載 →0.0；mps_req=50%→最高懲罰≈-1.0
Runtime fit	$\varepsilon = 0.00$	$(1 - pred_seconds)/fallback(14400s)$ 。 predictor 不可用=0.5；鼓勵短工作先排。

- SAC：忠實實作 Haarnoja, T., et al. [15] 的離散版本，純量 twin-Q critic，無分布式 critic 或風險機制。
- RDSAC：忠實實作 Ma, X., et al. [14] 的離散版本，雙分布 IQN critic 加風險扭曲，提供 mean（風險中立）與 cvar（，下尾風險敏感）兩種模式。

3.4.1 訓練與評估管線

直接在實際環境從頭訓練的話，強化學習需要數十萬到數百萬個 transition，而真實叢集一個編排決策對應一個跑數分鐘至數小時的任務，湊滿樣本要等數月，因此採 sim-to-real 兩段式：

- (1) 在模擬環境大量訓練，產出基本模型 (checkpoint)
- (2) 上線部署，記錄真實叢集 (observation, action, reward) 資料
- (3) RLPD (Reinforcement Learning with Prior Data) [24] 用真實資料把模擬的基本模型微調成真實環境策略

3.4.2 評估指標

在評估一個排程策略時都有主要指標平均工作完成時間(mean JCT)和尾部延遲指標 p95/p99 JCT、CVaR(0.25)。因為從 mean JCT 難以看出把 straggler、queue starvation、head-of-line blocking 等問題，即「多數 job 正常、少數被拖很慢」的情況。這些慢 job 幾乎不影響 mean JCT，但主導使用者體感。而 p95/p99 指標專門抓「最差 5%/1% 有多慢」，正是 mean 結構上看不到的那段。此外，RDSAC-cvar 的設計目標就是優化回報下尾(= JCT 上尾)；只測量 mean 等於拿不會動的尺去量專門改尾部的的方法，結構上必然測不出差異。

3.4.3 模擬配對基準測試

使用相同亂數種子(seed)對每種排程法做配對比較，降低 trace 隨機性造成的誤差。其中 $\Delta JCT\% = \frac{score-model}{score} \times 100\%$ ，負值代表模型的 JCT 較高、較差。每個 trace 取 50 個 job，訓練 100,000 步(含 curriculum n_jobs 10→30→50，fixed- $\alpha=0.05$)。評估工作在兩個工作負載家族 Philly [26] 和 Alibaba [27]、各 5 個亂數種子下進行。

3.4.4 實機 A/B 測試

在實驗室環境(RTX 4070 + RTX 3080)提交 cuBLAS sgemm [25]工作，做配對 A/B 測試。驗證以 3 個訓練亂數種子(42/43/44)各跑一次四方 A/B，確保對訓練種子穩健。每個 GPU 工作用同一條 stream 在四種方法(score/SAC/RDSAC-mean/RDSAC-cvar)各重放一次。並且減少叢集飄移(drift)的偏差，即 GPU 跑久了會變快。若一種方法整段跑完才換下一種，drift 會和方法混淆，故改用 Round Robin 交錯方法順序、每方法跨多輪並輪換各個位置，把飄移誤差平均掉。

第四章 結果與討論

4.1 平台建置成果

目前系統已完成主要功能建置，並通過實機驗證。主要成果如下：

4.1.1 平台建置與操作

已完成主要平台建置，並整理出清楚的部署與驗證流程如下表 2。使用者可透過登入節點提交工作，系統則自動在背後完成節點啟動、工作排程、GPU 分配與結果輸出。

【表 2】預期工作項目與達成狀態

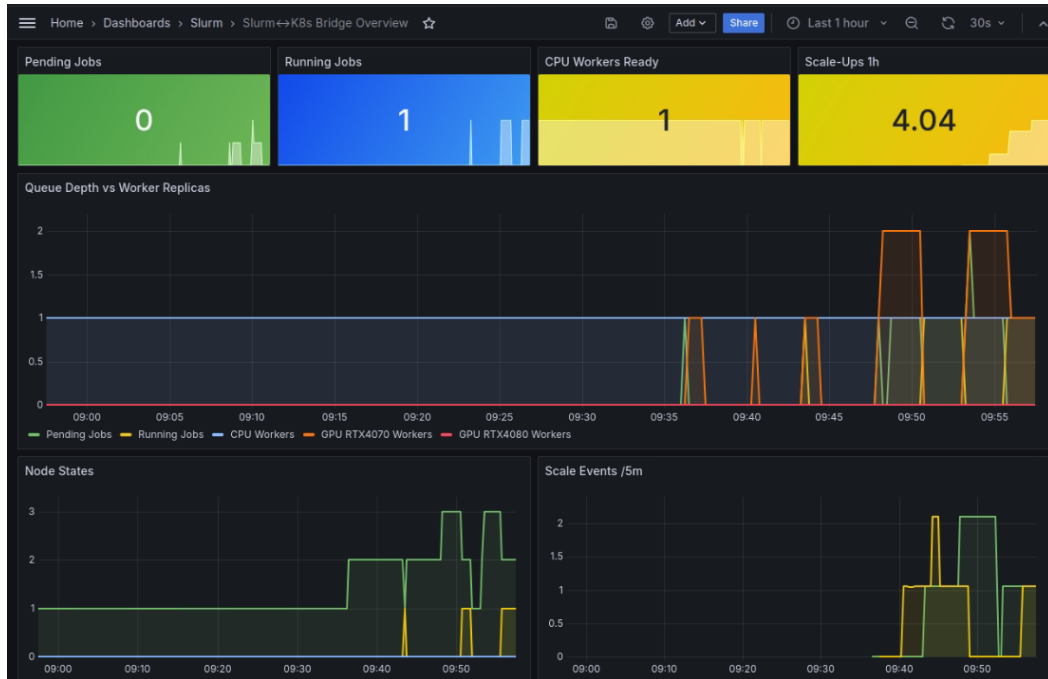
工作項目	預期目標	目前狀態
CPU/GPU 資源自動調整	依工作量自動擴縮節點	已完成
系統監控與視覺化	可觀察佇列、節點狀態與 GPU 使用	已完成
啟發式基準排程	建立穩定的啟發式 score 基準	已完成
SAC 與 RDSAC 排程	已上線	已完成
實機 A/B 測試架構	配對比較 DRL vs. 啟發式	已完成
RLPD	用真實資料把模擬的基本模型微調成真實環境策略	未完成

4.1.2 監控與展示介面

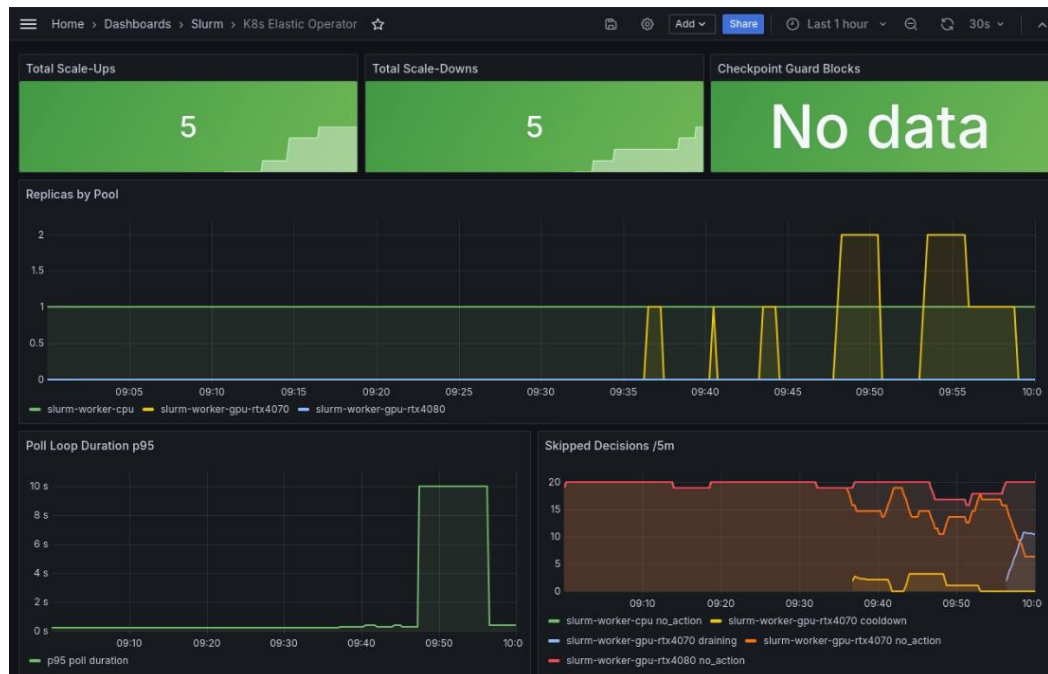
目前系統以 Prometheus [7] 統一收集 Slurm 佇列與節點狀態、Kubernetes StatefulSet 與 Pod 狀態、彈性 Operator 事件、GPU 指標，以及 RL Scheduler 決策資訊，其中 Kubernetes 資源狀態透過 kube-state-metrics [13] 補齊，再由 Grafana [11] 整理成多個儀表板如下：

- 圖 4 的系統橋接儀表板用來呈現 Slurm 語意如何驅動 Kubernetes 行為，包含 job 數目、擴縮次數、資源利用率、Queue 深度等。
- 圖 5 的 K8s Operator Dashboard 用來觀察彈性伸縮控制器的行為，包含 scale-up 與 scale-down 事件時間線，以及各資源池如 CPU 與 GPU worker pool 的 replica count 變化。
- 圖 6 的 GPU 資源儀表板直接使用 NVIDIA DCGM exporter 的硬體指標，顯示各 GPU 的 SM 使用率、VRAM 已用與可用容量、Memory Copy Util、GPU 溫度、板卡功耗，以及目前每張 GPU 的即時利用率與 VRAM 使用量；可補足 Slurm 只知道 GPU 是否被分配、但不知道硬體是否真正忙碌的限制。

- 圖 7 的排程資訊儀表板是給使用者觀察智慧排程流向的即時看板，包含最近選到的 job/node/GPU、policy value、policy entropy、可用 MPS slot、GPU 使用率等。



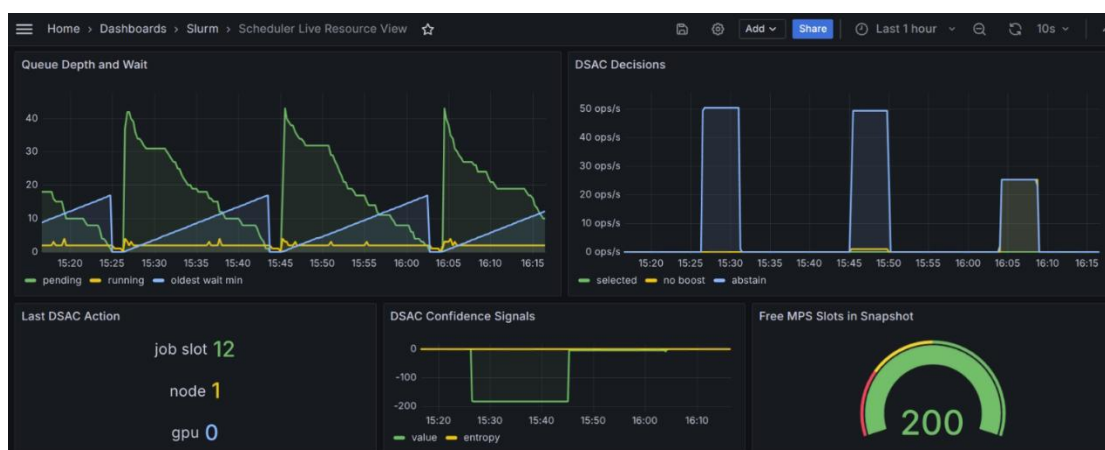
【圖 4】系統橋接儀表板



【圖 5】系統橋接儀表板



【圖 6】GPU 資源儀表板



【圖 7】排程資訊儀表板

4.2 評估結果

在模擬環境和實機環境的評估結果如下。

4.2.1 模擬環境評估

模擬評估在三個訓練亂數種子，使用和啟發式 score 的 JCT 差距百分比如下表 3，結果顯示沒有 DRL 演算法可以穩定超越 score：

【表 3】全方法的模擬環境評估結果

資料集	排程	JCT	p95	p99	Δ JCT%
philly	SAC	2.3±0.1	11.4±0.5	28.4±1.5	-11.8±4.2
philly	RDSAC-mean	2.6±0.1	11.7±1.2	41.0±8.8	-23.1±7.4
philly	RDSAC-cvar	2.1±0.1	9.3±1.1	30.4±3.8	-4.1±4.6
ali	SAC	1.2±0.1	5.5±1.2	21.1±0.3	-15.2±17.5
ali	RDSAC-mean	1.4±0.1	6.6±0.5	22.7±2.5	-32.7±10.4
ali	RDSAC-cvar	1.1±0.1	4.3±0.9	20.4±0.6	-12.0±12.0

4.2.2 實機 A/B 測試結果

在實機上評估四種排程方法的差異，拿三個亂數種子(42,43,44)來做整合評估。參考下表 4，可以發現 Score 略勝 DRL 方法，而 DRL 策略的排名大致為 RDSAC-mean>RDSAC-cvar>SAC。

【表 4】全方法的實機評估結果 (mean±std)

排程／指標	JCT	p95	p99	CVaR	Δ JCT%
Score	169.4±1.4	398.9±0.9	429.8±3.7	375.8±1.1	-
SAC	175.6±1.4	403.9±1.6	431.2±1.9	381.7±1.9	-3.6±0.4
RDSAC-mean	173.5±0.9	399.3±3.2	430.0±5.1	377.7±3.1	-2.4±1.4
RDSAC-cvar	175.3±0.3	404.7±0.9	430.4±3.6	381.9±0.5	-3.4±1.0

4.3 結論

目前已完成以 Slurm-on-Kubernetes 為核心的共享 GPU AI 工作負載彈性排程平台，整合工作提交、自定義 DRL 決策模組、Kubernetes 節點管理、MPS GPU 資源切分和 Prometheus 監控等技術，使系統能在實機環境中自動處理工作排序、節點啟動、GPU 配置與狀態追蹤。實驗結論顯示，平台功能已能穩定運作，DRL 客製化排程插件也能正確參與實機決策；然而在目前的模擬評估與實機 A/B 測試中，SAC 與 RDSAC 等 DRL 方法尚未穩定超越啟發式 score，表示本階段的主要貢獻在於完成可實測的平台整合與建立後續演算法比較基準。

未來工作將優先補強 FCFS、SJF 等基本 baseline，擴充更多真實工作負載與長時間叢集資料，並導入 RLPD 以真實觀測資料微調模擬訓練所得模型，降低 sim-to-real 落差。後續也可進一步改善模型特徵、獎勵設計與風險參數設定，加入更完整的 GPU 拓樸、工作執行時間預估與多使用者公平性考量，使智慧排程策略在維持穩定性的同時，有機會在平均工作完成時間與尾部延遲上優於啟發式方法。

第五章 參考文獻

1. SchedMD. (n.d.). *Slurm workload manager documentation*. <https://slurm.schedmd.com/>
2. SchedMD. (n.d.). *Slurm plugin API*. <https://slurm.schedmd.com/plugins.html>
3. The Kubernetes Authors. (n.d.). *Kubernetes documentation*. <https://kubernetes.io/docs/>
4. *Converged computing: Integrating HPC and cloud native*. (2024). IEEE Computer Society. <https://www.computer.org/csdl/magazine/cs/2024/03/10770850/22fgId5NFpC>
5. *Kubernetes v1.35: Introducing workload-aware scheduling*. (2025, December 29). The Kubernetes Blog. <https://kubernetes.io/blog/2025/12/29/kubernetes-v1-35-introducing-workload-aware-scheduling/>
6. SchedMD. (n.d.). *Slinky* [Computer software]. GitHub. <https://github.com/slinkyproject>
7. Penso, V. (n.d.). *Prometheus Slurm exporter* [Computer software]. GitHub. <https://github.com/vpenso/prometheus-slurm-exporter>
8. Amazon Web Services. (n.d.). *AWS ParallelCluster* [Computer software]. GitHub. <https://github.com/aws/aws-parallelcluster>
9. Texas Advanced Computing Center. (n.d.). *Lmod: An environment module system* [Computer software]. GitHub. <https://github.com/TACC/Lmod>
10. The Kubernetes Authors. (n.d.). *Scheduler configuration: Scoring*. Kubernetes. <https://kubernetes.io/docs/reference/scheduling/config/>
11. Grafana Labs. (n.d.). *Grafana*. <https://grafana.com/>
12. NVIDIA. (n.d.). *Multi-Process Service (MPS)*. NVIDIA Documentation. <https://docs.nvidia.com/deploy/mps/>
13. The Kubernetes Authors. (n.d.). *kube-state-metrics* [Computer software]. GitHub. <https://github.com/kubernetes/kube-state-metrics>
14. Ma, X., Xia, L., Zhou, Z., Yang, J., & Zhao, Q. (2020). *DSAC: Distributional soft actor-critic for risk-sensitive reinforcement learning* [Preprint]. arXiv. <https://arxiv.org/abs/2004.14547>
15. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/1801.01290>
16. Christodoulou, P. (2019). *Soft actor-critic for discrete action settings* [Preprint]. arXiv. <https://arxiv.org/abs/1910.07207>
17. Dabney, W., Ostrovski, G., Silver, D., & Munos, R. (2018). Implicit quantile networks for distributional reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/1806.06923>

18. *Reinforcement learning for AI as a service: CPU–GPU task scheduling for preprocessing, training, and inference tasks*. (n.d.). IEEE.
<https://ieeexplore.ieee.org/abstract/document/10976623>
19. Abdul Majeed, A., et al. (n.d.). *Scheduling techniques of AI models on modern heterogeneous edge GPU—A critical review*. IEEE.
<https://ieeexplore.ieee.org/abstract/document/11354153>
20. Lipe, E., et al. (n.d.). *Energy-efficient scheduling of AI/ML workloads on multi-instance GPUs with dynamic repartitioning*. IEEE.
<https://ieeexplore.ieee.org/abstract/document/11044810>
21. Sedighi, H., et al. (n.d.). *Dynamic task scheduling and adaptive GPU resource allocation in the cloud*. IEEE.
<https://ieeexplore.ieee.org/abstract/document/11261404>
22. Wang, X., et al. (n.d.). *Dynamic GPU scheduling with multi-resource awareness and live migration support*. IEEE.
<https://ieeexplore.ieee.org/abstract/document/10091187>
23. Chab, R., et al. (2025). Algorithmic techniques for GPU scheduling: A comprehensive survey. *Algorithms*, 18(7), 385. <https://www.mdpi.com/1999-4893/18/7/385>
24. Philip J. Ball, et al. (2023). *Efficient Online Reinforcement Learning with Offline Data* [Preprint]. arXiv. <https://arxiv.org/abs/2302.02948>
25. NVIDIA. (n.d.). *cuBLAS Introduction*. NVIDIA Documentation.
<https://docs.nvidia.com/cuda/cublas/>
26. Microsoft Research. (n.d.). *Philly traces* [Data set]. GitHub.
<https://github.com/msr-fiddle/philly-traces>
27. Alibaba. (n.d.). *Cluster data: Production cluster traces* [Data set]. GitHub.
<https://github.com/alibaba/clusterdata>